

UNIT-II

Regular Expression (RE): Definition, Operators of regular expression and their precedence, Algebraic laws for Regular expressions, Kleen's Theorem, Regular expression to FA, DFA to Regular expression, Arden's Theorem, Non-Regular Languages, Pumping Lemma for Regular Languages. Application of Pumping Lemma, Closure properties of Regular Languages, Decision properties of Regular Languages, FA with output: Moore and Mealy machine, Equivalence of Moore and Mealy Machine, Applications and Limitation of FA.

Regular Expressions, Regular Grammar and Regular Languages

Regular Expressions

Regular expressions are symbolic notations used to define search patterns in strings. They describe **regular languages** and are commonly used in tasks such as validation, searching, and parsing.

A **regular expression** represents a **regular language** if it follows these rules:

1. **ϵ (epsilon) is a regular expression** representing the language **$\{\epsilon\}$** (the empty string).
2. **Any symbol 'a' from the input alphabet Σ is a regular expression**, representing the language **$\{a\}$** .
3. **Union ($a + b$) is a regular expression** if **a** and **b** are regular expressions, representing the language **$\{a, b\}$** .
4. **Concatenation (ab) is a regular expression** if **a** and **b** are regular expressions.
5. *Kleene star (a) is a regular expression^{*}*, meaning **zero or more occurrences of 'a'**, forming a regular language.

Description	Regular Expression	Regular Languages
Set of vowels	<code>`(a</code>	e
'a' followed by 0 or more 'b'	<code>ab*</code>	{a, ab, abb, abbb, abbbb, ...}
Any number of vowels followed by any number of consonants	<code>[aeiou]*[bcdfghjklmnpqrstvwxyz]*</code>	{ ϵ , a, aou, aiou, b, abcd, ...} (ϵ represents empty string)

Regular Grammar

A regular grammar is a formal grammar that generates regular languages. It consists of:

- **Terminals:** Symbols that form strings (e.g., a, b).
- **Non-terminals:** Variables used to derive strings (e.g., S, A).

- **Production Rules:** Rules for transforming non-terminals into terminals or other non-terminals.
- **Start Symbol:** The non-terminal from which derivations begin.

Types of Regular Grammar

1. **Right-linear Grammar:** All production rules are of the form: $A \rightarrow aB$ or $A \rightarrow a$.

Example:

$S \rightarrow aS \mid bS \mid \epsilon$

2. **Left-linear Grammar:** All production rules are of the form: $A \rightarrow Ba$ or $A \rightarrow a$.

read more about - [Regular grammar](#)

Regular Languages

Regular languages are the class of languages that can be represented using finite automata, regular expressions, or regular grammar. These languages have predictable patterns and are computationally efficient to recognize.

Properties of Regular Languages

1. Closure Properties

Regular languages are closed under operations like union, concatenation, and Kleene star.

- **Union:** If L_1 and L_2 are two regular languages, their union $L_1 \cup L_2$ will also be regular. For example, $L_1 = \{a^n \mid n \geq 0\}$ and $L_2 = \{b^n \mid n \geq 0\}$ $L_3 = L_1 \cup L_2 = \{a^n \cup b^n \mid n \geq 0\}$ is also regular.
- **Intersection:** If L_1 and L_2 are two regular languages, their intersection $L_1 \cap L_2$ will also be regular. For example, $L_1 = \{a^m b^n \mid n \geq 0 \text{ and } m \geq 0\}$ and $L_2 = \{a^m b^n \mid b^n a^m \mid n \geq 0 \text{ and } m \geq 0\}$ $L_3 = L_1 \cap L_2 = \{a^m b^n \mid n \geq 0 \text{ and } m \geq 0\}$ is also regular.
- **Concatenation:** If L_1 and L_2 are two regular languages, their concatenation $L_1.L_2$ will also be regular. For example, $L_1 = \{a^n \mid n \geq 0\}$ and $L_2 = \{b^n \mid n \geq 0\}$ $L_3 = L_1.L_2 = \{a^m.b^n \mid m \geq 0 \text{ and } n \geq 0\}$ is also regular.
- **Kleene Closure:** If L_1 is a regular language, its Kleene closure L_1^* will also be regular. For example, $L_1 = \{a \cup b\}$ $L_1^* = (a \cup b)^*$
- **Complement:** If $L(G)$ is regular language, its complement $L'(G)$ will also be regular. Complement of a language can be found by subtracting strings which are in $L(G)$ from all possible strings. For example, $L(G) = \{a^n \mid n > 3\}$ $L'(G) = \{a^n \mid n \leq 3\}$

read more about - [Closure properties of Regular languages](#)

2. Finite Automata:

Every regular language can be recognized by a finite automaton (DFA or NFA).

3. Decision Problems:

Problems like membership testing, emptiness, and equivalence can be solved for regular languages.

Note: Two regular expressions are equivalent if languages generated by them are same. For example, $(a+b^*)^*$ and $(a+b)^*$ generate same language. Every string which is generated by $(a+b^*)^*$ is also generated by $(a+b)^*$ and vice versa.

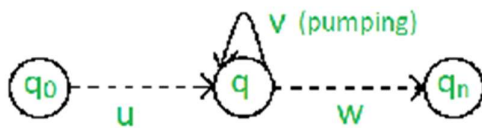
Comparison of Regular Expressions, Grammar, and Languages

Aspect	Regular Expressions	Regular Grammar	Regular Languages
Definition	Pattern representation of strings	Rule-based generation of strings	Language class described by regex and grammar
Representation	Symbols and operators	Terminals, non-terminals, production rules	Finite automata, regex, or grammar
Use Case	Text processing, validation	Syntax generation for compilers	Language recognition

Pumping Lemma in Theory of Computation

There are two Pumping Lemmas, which are defined for 1. Regular Languages, and 2. Context - Free Languages

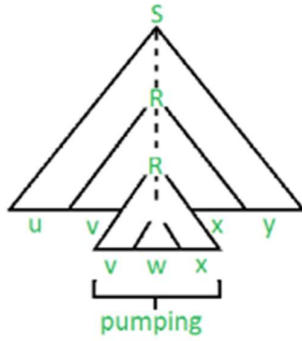
Pumping Lemma for Regular Languages For any regular language L , there exists an integer n , such that for all $x \in L$ with $|x| \geq n$, there exists $u, v, w \in \Sigma^*$, such that $x = uvw$, and (1) $|uv| \leq n$ (2) $|v| \geq 1$ (3) for all $i \geq 0$: $uv^i w \in L$ In simple terms, this means that if a string v is 'pumped', i.e., if v is inserted any number of times, the resultant string still remains in L . Pumping Lemma is used as a proof for irregularity of a language. Thus, if a language is regular, it always satisfies pumping lemma. If there exists at least one string made from pumping which is not in L , then L is surely not regular. The opposite of this may not always be true. That is, if Pumping Lemma holds, it does not mean that the language is regular.



For example, let us prove $L_0 = \{0^n 1^n \mid n \geq 0\}$ is irregular. Let us assume that L is regular, then by Pumping Lemma the above given rules follow. Now, let $x \in L$ and $|x| \geq n$. So, by Pumping Lemma, there exists u, v, w such that (1) – (3) hold. We show that for all u, v, w , (1) - (3) does not hold. If (1) and (2) hold then $x = 0^n 1^n = uvw$ with $|uv| \leq n$ and $|v| \geq 1$. So, $u = 0^a, v = 0^b, w = 0^c 1^n$ where: $a + b \leq n, b \geq 1, c \geq 0, a + b + c = n$ But, then (3) fails for $i = 0$ $uv^0 w = uw = 0^a 0^c 1^n = 0^{a+c} 1^n \notin L$, since $a + c < n$.



Pumping Lemma for Context-free Languages (CFL) Pumping Lemma for CFL states that for any Context Free Language L , it is possible to find two substrings that can be 'pumped' any number of times and still be in the same language. For any language L , we break its strings into five parts and pump second and fourth substring. Pumping Lemma, here also, is used as a tool to prove that a language is not CFL. Because, if any one string does not satisfy its conditions, then the language is not CFL. Thus, if L is a CFL, there exists an integer n , such that for all $x \in L$ with $|x| \geq n$, there exists $u, v, w, x, y \in \Sigma^*$, such that $x = uvwxy$, and (1) $|vwx| \leq n$ (2) $|vx| \geq 1$ (3) for all $i \geq 0$: $uv^i w x^i y \in L$



For above example, $0^n 1^n$ is CFL, as any string can be the result of pumping at two places, one for 0 and other for 1. Let us prove, $L_{012} = \{0^n 1^n 2^n \mid n \geq 0\}$ is not Context-free. Let us assume that L is Context-free, then by Pumping Lemma, the above given rules follow. Now, let $x \in L$ and $|x| \geq n$. So, by Pumping Lemma, there exists u, v, w, x, y such that (1) – (3) hold. We show that for all u, v, w, x, y (1) - (3) do not hold. If (1) and (2) hold then $x = 0^n 1^n 2^n = uvwxy$ with $|vwx| \leq n$ and $|vx| \geq 1$. (1) tells us that vwx does not contain both 0 and 2. Thus, either vwx has no 0's, or vwx has no 2's. Thus, we have two cases to consider. Suppose vwx has no 0's. By (2), vx contains a 1 or a 2. Thus uwy has ' n ' 0's and uwy either has less than ' n ' 1's or has less than ' n ' 2's. But (3) tells us that $uwy = uv^0wx^0y \in L$. So, uwy has an equal number of 0's, 1's and 2's gives us a contradiction. The case where vwx has no 2's is similar and also gives us a contradiction. Thus L is not context-free. Source : John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman (2003). Introduction to Automata Theory, Languages, and Computation.

The pumping lemma is one of the basic tools in automata theory and it is used for proving properties for regular and context-free languages. In this chapter, we will cover the most significant applications of Pumping Lemma in computer science and linguistics.

What is Pumping Lemma?

For a basic recap, the pumping lemma holds in regular and context-free languages. And it provides a way to prove that a language is not in one of these classes by showing its strings can be "pumped" or repeated in some way while still being in the language.

Applications of Pumping Lemma

Some of the key applications of the Pumping Lemma are highlighted below with a brief description on each of them.

1. Proof of Non-Regularity of Languages

One of the most common uses of the Pumping Lemma is in language classes, where it is used to prove the irregularity of a certain language by highlighting that no matter which division of a string in that language is made, the conditions given by the Pumping Lemma could never be satisfied. For a basic understanding, take a look at the following example.

Example

Consider the language $L = \{a^n b^n \mid n \geq 0\}$.

Assume L is regular, then let p be the pumping length. And choose, $s = a^p b^p$.

Divide $s = xyz$, with xy containing no more than p symbols.

By pumping y , we get $xy^2z = a^{p+i}b^p$, which is not in L since the number of a 's and b 's are not equal.

Thus, **L is not regular.**

2. Proving Non-Context-Free Nature of Languages

The Pumping Lemma for context-free languages is used to show that a language is not context-free. This involves demonstrating that the language cannot satisfy the lemma's conditions.

Example

Consider the language, $L = \{a^n b^n c^n \mid n \geq 0\}$, now assume L is context-free. And let p be the pumping length. So, select $s = a^p b^p c^p$.

Divide $s = uvwx^y$, ensuring vw contains no more than p symbols.

By pumping v and x , the structure $a^n b^n c^n$ is violated as the equal number of a 's, b 's, and c 's will not be maintained.

Thus, **L is not context-free.**

3. Understanding Language Structure

The Pumping Lemma is useful in analysing structure in languages. In the context of the information on how strings could be pumped, it may be possible that the researchers could, in turn, why get some insights of the repetitive patterns and underlying properties of the languages.

4. Designing Automata

In the design of finite automata, the pumping lemma helps check whether a proposed automaton can accept some given language.

If some language cannot pass the Pumping Lemma, then it means that it cannot recognize that language, and thus, a finite automaton cannot be suitable for it.

5. Simplifying Language Classes

It helps classify and simplify such language classes through clear criteria in determining whether language is a regular or context-free one. After understanding which language it is, we can easily design the machine or model for them.

6. Algorithm Development

Another application could be algorithms incorporating language recognition and its processing can benefit greatly from the use of the Pumping Lemma. It can be used by developers to write effective algorithms for several language processing tasks. These will ensure that only regular or context-free languages are considered appropriate if they are so.

Conclusion

Pumping Lemma finds its application in many areas of automata theory and formal languages: proving non-regularity or non-context-free nature, understanding the structure of languages.

In this chapter, we highlighted the applications of Pumping Lemma when we can construct an automaton, simplifying language classes, and developing algorithms, etc. Because of its importance in theoretical computer science and linguistics, Pumping Lemma becomes the core concept in the field.

Kleene's Theorem establishes the equivalence between finite automata, regular expressions, and transition graphs, providing a foundational framework for understanding regular languages.

Overview of Kleene's Theorem

Kleene's Theorem is a fundamental concept in the Theory of Computation that asserts the following three statements are equivalent:

1. **Finite Automata (FA):** A language accepted by a finite automaton can also be accepted by a transition graph.
2. **Transition Graphs (TG):** A language accepted by a transition graph can be expressed by a regular expression.
3. **Regular Expressions (RE):** A language defined by a regular expression can be accepted by a finite automaton.
This theorem highlights the close relationship between these three representations of regular languages, showing that they can be converted into one another without loss of information.

3

4. 



3 Sources

Implications of Kleene's Theorem

- **Regular Languages:** The theorem implies that any language that can be recognized by a finite automaton is regular, and conversely, any regular language can be represented by a finite automaton or a regular expression.
- **Constructive Proof:** The proof of Kleene's Theorem is constructive, meaning that for any regular expression, one can construct a corresponding finite automaton that recognizes the same language. This is often done using methods such as state elimination or constructing nondeterministic finite automata (NFA) from regular expressions.

1

- 

1 Source

Applications

Kleene's Theorem is widely used in various fields, including:

- **Compiler Design:** For lexical analysis, where regular expressions are used to define tokens.
- **Automata Theory:** [To prove properties of regular languages and to design algorithms for converting between different representations \(e.g., from regular expressions to finite automata\).](#)

2

[In summary, Kleene's Theorem serves as a cornerstone in the study of formal languages and automata, providing essential insights into the nature of regular languages and their representations. For further reading, you can explore detailed explanations and examples in the provided sources](#)